Day 14 圖解:CI 與測試

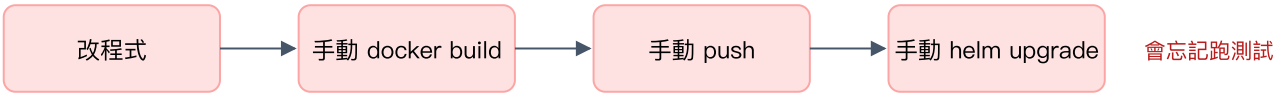
GitLab CI pipeline + 後端 pytest 測試 · 專案:Chatbot on AKS

今天做了什麼(一句話)

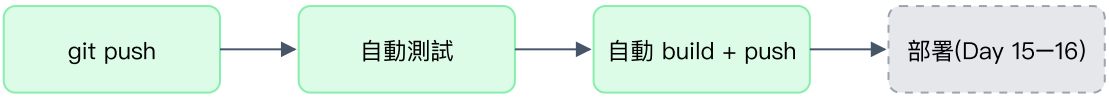
讓**機器在每次 push 時自動跑測試、自動 build image**;測試只要有一個失敗,就不會產生 image,壞掉的版本進不了後面的部署流程。

之前 vs 現在

之前:全靠人手



現在:push 就自動跑



今天的產出

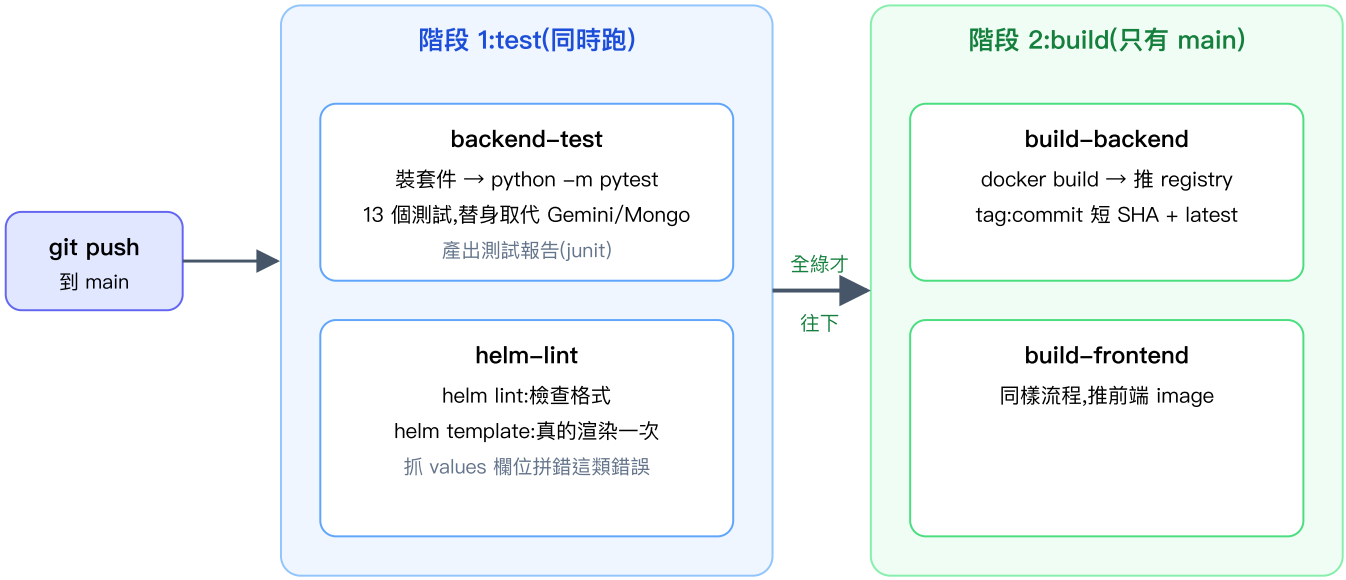
項目	位置	用途
13 個後端測試	`chatbot-backend/tests/`	檢查 API 行為與失敗路徑
測試設定	`pytest.ini`、`requirements-dev.txt`	告訴 pytest 去哪找測試、需要哪些套件
CI 設定	`.gitlab-ci.yml`	定義 pipeline 的階段與工作
.dockerignore 調整	`chatbot-backend/.dockerignore`	測試檔不要被打包進正式 image

驗證結果

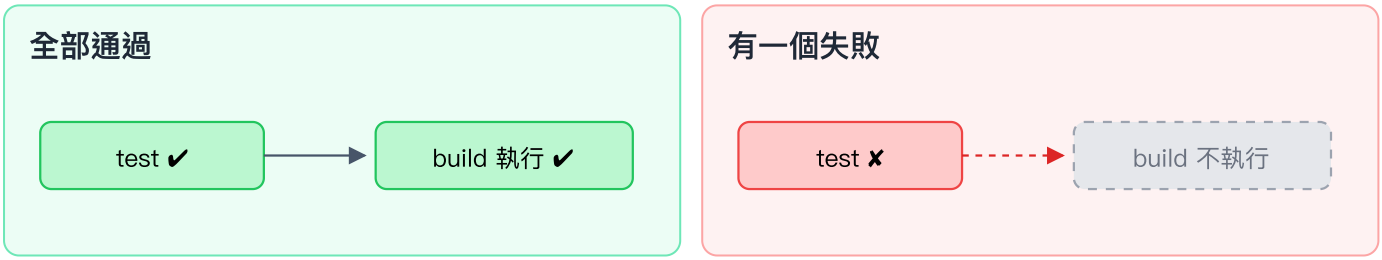
- 本機:13 個測試 0.09 秒全過。
- GitLab:pipeline 全綠(`backend-test`、`helm-lint`、兩個 build)。
- 故意推 `assert 1 + 1 == 3`:`backend-test` 變紅,**兩個 build 沒有執行**。這是「測試失敗擋住發版」的實證。

Pipeline 怎麼運作

每次 push,GitLab 讀 `.gitlab-ci.yml`,開一台臨時機器照著跑



為什麼測試紅了 build 就不跑?



規則由 GitLab 提供:同一個 pipeline 中,前一階段只要有任何 job 失敗,後面的階段就不會跑。我們只是把「測試」放在「build」前面。

關鍵設定摘錄

```
stages: [test, build]              # 順序就是擋關的關鍵

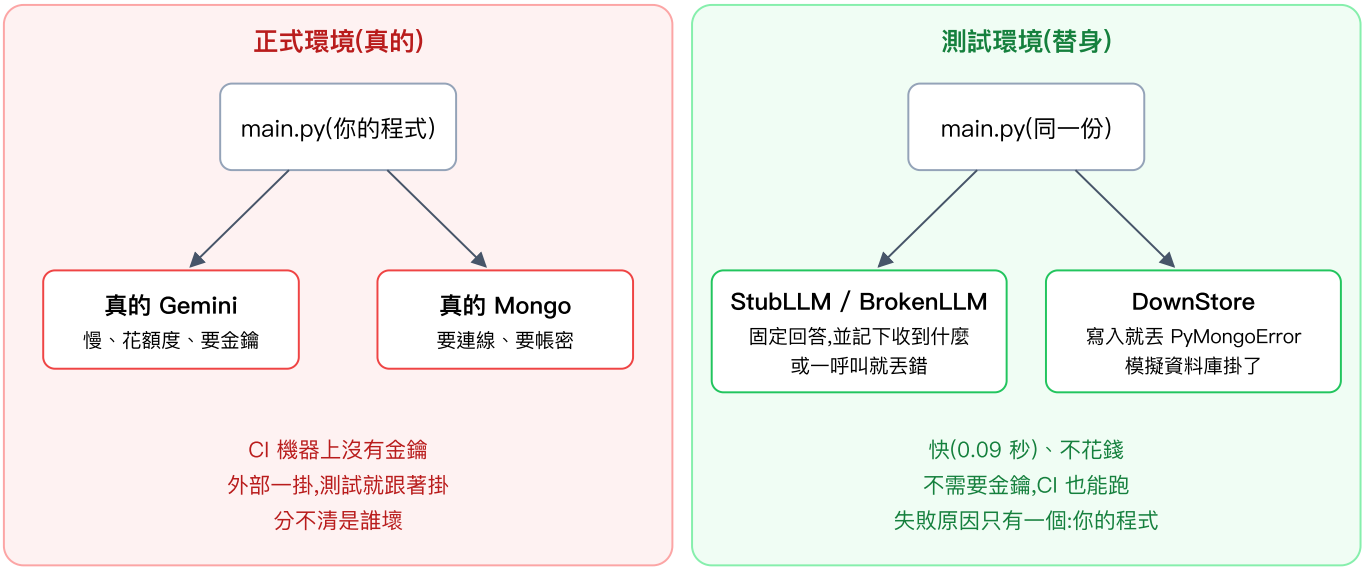
build-backend:
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH   # 只有 main 才 build
  script:
    - docker build -t $IMAGE:$CI_COMMIT_SHORT_SHA ...
    - docker push  $IMAGE:$CI_COMMIT_SHORT_SHA
```

為什麼用 commit 短 SHA 當 tag?每個 image 都能對回某一次 commit,出事時能知道正在跑的是哪一版程式碼。`latest` 會一直變,不能用來追溯。

registry 登入不用自己設密碼: GitLab 自動注入 `CI_REGISTRY_USER` / `CI_REGISTRY_PASSWORD`,所以沒有任何 secret 放在 repo 或需要你保管。

測試的核心觀念:替身

測試不能打真的 Gemini 和 Mongo,所以用「假的」換掉它們



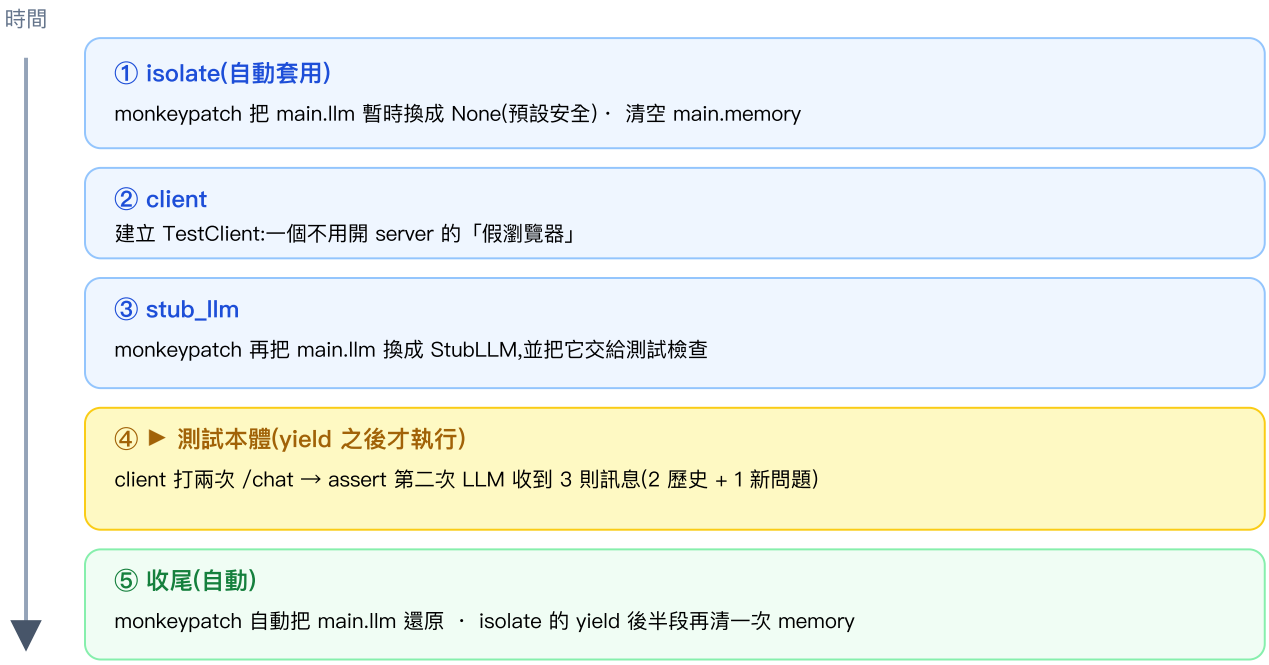
三個替身各自負責什麼

替身	行為	用來測
`StubLLM`	回固定答案,並把每次收到的訊息記在 `calls`	對話記憶、隔離、token 記錄
`BrokenLLM`	一呼叫就丟錯	Gemini 掛掉時回 502
`DownStore`	`append_turn` 丟 PyMongoError	Mongo 掛掉時回 503

替身的代價(面試要誠實講):替身測得到「你的邏輯對不對」,測不到「跟真的 Gemini/Mongo 整合會不會出問題」。那一塊要靠 Day 22–24 在真的 cluster 上做整合測試補。

fixture 與 monkeypatch:一個測試的一生

以 `test_chat_remembers_history(client, stub_llm)` 為例



fixture

測試前的「準備工作」寫成函式,貼上 `@pytest.fixture`。測試的參數寫上名字,pytest 就自動先執行它。

```
def test_health(client):   # 參數名 = fixture 名
    client.get("/health")
```

monkeypatch

測試期間**暫時**換掉程式裡的某個東西,測完**自動還原**,不影響其他測試。

```
monkeypatch.setattr(
    main,   # 哪個模組
    "llm",  # 換哪個變數
    None)   # 換成什麼
```

`yield` 是分隔線:在 fixture 裡,yield 之前是「測試前準備」,之後是「測試後收尾」。
`autouse=True`:不用在參數寫名字,每個測試都自動套用(isolate 用它,當作保險)。

為什麼要清 memory? `main.memory` 是全域字典,上一個測試的對話會殘留。測試之間若互相影響,會出現「單獨跑會過、一起跑會失敗」這種最難找的 bug。

13 個測試在檢查什麼

好的測試是在釘住「你不想讓它壞第二次」的行為

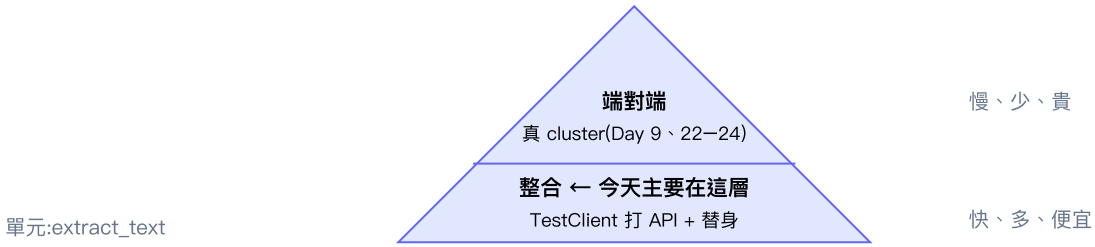
API 行為(`test_api.py`,9 個)

測試	檢查	為什麼重要
health	回 200	K8s 探針依賴它
沒有 LLM 時	回 `[mock reply]` 並產生 conversation_id	沒金鑰也不會爆
缺 message	回 422	輸入驗證
同一對話記得歷史	第二輪 LLM 收到 3 則	對話記憶是核心功能
對話互相隔離	不同使用者不串線	隱私
記憶有上限	不超過 20 則	防記憶體與 token 暴增
LLM 失敗	回 502,**且失敗那輪不留在記憶**	否則重試時歷史被汙染
Mongo 失敗	回 503,**且記憶不更新**	保證「先寫 Mongo 再更新記憶體」
extract_text	字串與區塊清單都能處理	新模型回傳格式不固定

Metrics(`test_metrics.py`,4 個)

測試	檢查	為什麼重要
序列一開始就存在	值 0 的 502 序列已出現	Day 13 的 bug:否則告警算不出第一次增量
請求被計數	打一次 /chat,計數加 1	SLO 告警依賴它
亂打網址	全歸成 `unmatched`	防 Prometheus 時間序列爆量
token 有記錄	output 加 20	成本告警依賴它

測試的分層



面試怎麼講

30 秒版本:「我的 CI 分兩個階段:先跑測試與 Helm 檢查,通過才 build image 並推 registry。測試用替身隔離 Gemini 和 Mongo,所以 CI 不需要任何金鑰,13 個測試不到一秒。我實際演示過:故意推一個失敗的測試,test 階段變紅、build 階段完全沒執行,壞版本進不了後面的流程。」

可能被追問的問題

問題	回答重點
怎麼測 LLM 這種外部依賴?	用替身隔離;測的是自己的邏輯(重試、錯誤處理、記憶),不是 Gemini 本身
替身有什麼風險?	測不到真實整合問題,所以另外做整合測試(Day 22–24)
為什麼測試要快?	才能放進每次 push 的 CI;慢的測試最後會被人跳過
怎麼證明測試真的有用?	故意製造失敗,確認它會變紅、會擋住 build
測什麼最有價值?	失敗路徑(502/503/422),因為正式環境出事都在那裡
為什麼用 commit SHA 當 image tag?	可追溯,出事時知道跑的是哪一版

誠實的限制

- **前端沒有測試**,CI 只做 build。
- **只做到 CI,沒有自動部署**;部署是 Day 15–16 的 ArgoCD。
- **替身不能取代真整合測試**。
- AKS 要拉 GitLab registry 的 image 需要 imagePullSecret,或改推 ACR,Day 15 決定。

今天踩到的一個坑

還原壞測試時,`git revert` 沒生效,我又用 amend 改到錯的 commit,GitHub 曾短暫收到「訊息寫還原、檔案還在」的版本。沒有強制覆蓋,改用刪檔加 merge 接回,最終兩邊一致。**教訓:操作完一定要驗證結果**(我是看 `ls` 發現壞測試還在)。

接下來

